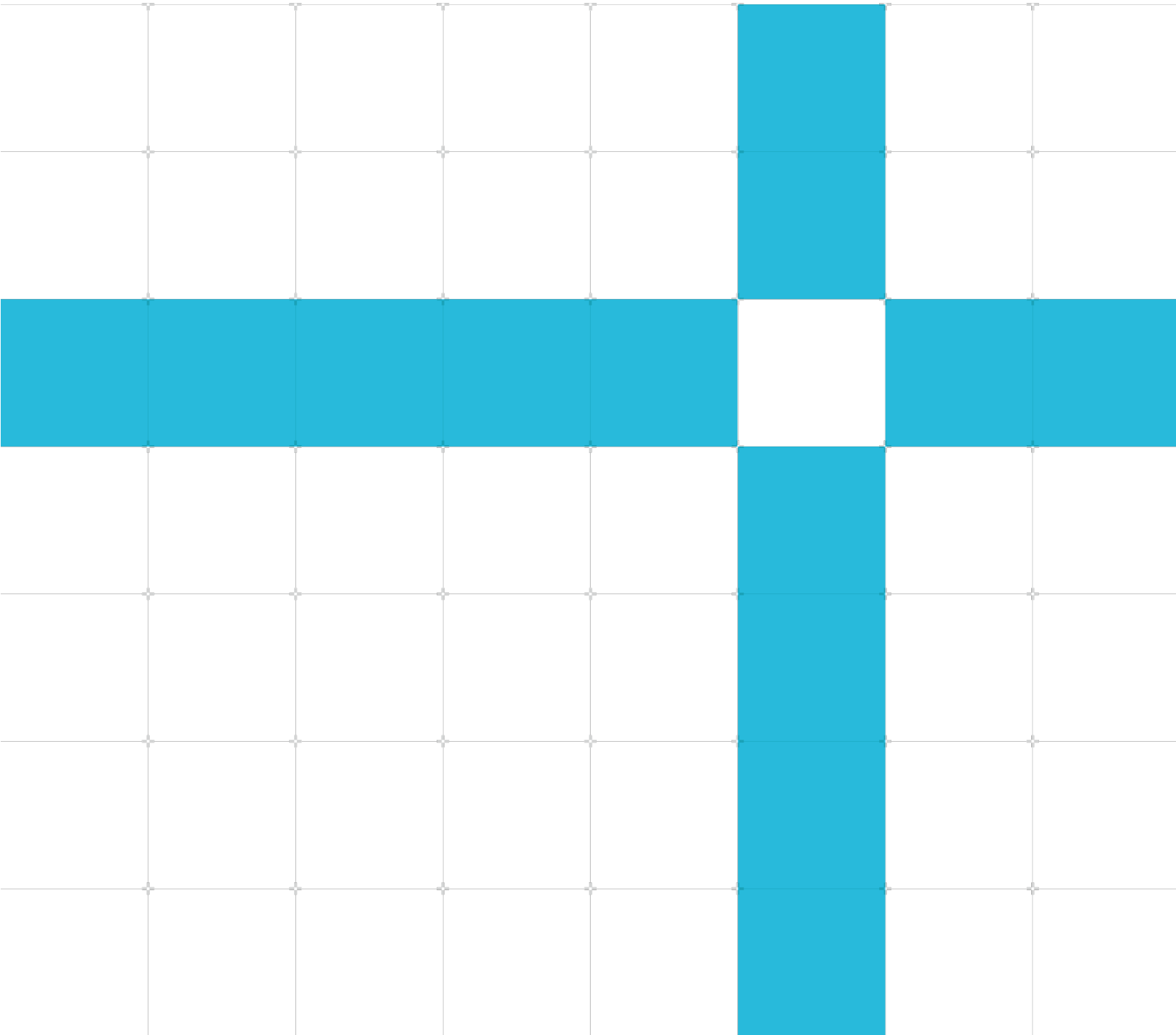
arm

SVE and Neon coding compared

Non-Confidential

Copyright © 2020 Arm Limited (or its affiliates).
All rights reserved.

Issue 0101
102131



SVE and Neon coding compared

Copyright © 2020 Arm Limited (or its affiliates). All rights reserved.

Release information

Document history

Issue	Date	Confidentiality	Change
01	14 September 2020	Non-Confidential	First release

Non-Confidential Proprietary Notice

This document is protected by copyright and other related rights and the practice or implementation of the information contained in this document may be protected by one or more patents or pending patent applications. No part of this document may be reproduced in any form by any means without the express prior written permission of Arm. No license, express or implied, by estoppel or otherwise to any intellectual property rights is granted by this document unless specifically stated.

Your access to the information in this document is conditional upon your acceptance that you will not use or permit others to use the information for the purposes of determining whether implementations infringe any third party patents.

THIS DOCUMENT IS PROVIDED "AS IS". ARM PROVIDES NO REPRESENTATIONS AND NO WARRANTIES, EXPRESS, IMPLIED OR STATUTORY, INCLUDING, WITHOUT LIMITATION, THE IMPLIED WARRANTIES OF MERCHANTABILITY, SATISFACTORY QUALITY, NON-INFRINGEMENT OR FITNESS FOR A PARTICULAR PURPOSE WITH RESPECT TO THE DOCUMENT. For the avoidance of doubt, Arm makes no representation with respect to, and has undertaken no analysis to identify or understand the scope and content of, patents, copyrights, trade secrets, or other rights.

This document may include technical inaccuracies or typographical errors.

TO THE EXTENT NOT PROHIBITED BY LAW, IN NO EVENT WILL ARM BE LIABLE FOR ANY DAMAGES, INCLUDING WITHOUT LIMITATION ANY DIRECT, INDIRECT, SPECIAL, INCIDENTAL, PUNITIVE, OR CONSEQUENTIAL DAMAGES, HOWEVER CAUSED AND REGARDLESS OF THE THEORY OF LIABILITY, ARISING OUT OF ANY USE OF THIS DOCUMENT, EVEN IF ARM HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

This document consists solely of commercial items. You shall be responsible for ensuring that any use, duplication or disclosure of this document complies fully with any relevant export laws and regulations to assure that this document or any portion thereof is not exported, directly or indirectly, in violation of such export laws. Use of the word "partner" in reference to Arm's customers is not intended to create or refer to any partnership relationship with any other company. Arm may make changes to this document at any time and without notice.

If any of the provisions contained in these terms conflict with any of the provisions of any click through or signed written agreement covering this document with Arm, then the click through or signed written agreement prevails over and supersedes the conflicting provisions of these terms. This document may be translated into other languages for convenience, and you agree that if there is any conflict between the English version of this document and any translation, the terms of the English version of the Agreement shall prevail.

The Arm corporate logo and words marked with ® or ™ are registered trademarks or trademarks of Arm Limited (or its affiliates) in the US and/or elsewhere. All rights reserved. Other brands and names mentioned in this document may be the trademarks of their respective owners. Please follow Arm's trademark usage guidelines at http://www.arm.com/company/policies/trademarks.

Copyright © 2020 Arm Limited (or its affiliates). All rights reserved.

Arm Limited. Company 02557590 registered in England.

110 Fulbourn Road, Cambridge, England CB1 9NJ.

(LES-PRE-20349)

Confidentiality Status

This document is Non-Confidential. The right to use, copy and disclose this document may be subject to license restrictions in accordance with the terms of the agreement entered into by Arm and the party that Arm delivered this document to.

Unrestricted Access is an Arm internal classification.

Web Address

www.arm.com

Contents

1 Overview **6**

2 Before you begin **7**

3 Data processing methodologies **8**

4 Fundamentals **9**
4.1 Instruction sets 9
4.2 Registers, vectors, lanes, and elements 10
4.3 Vector Length Agnostic programming 10

5 Coding best practices **12**

6 Compiler optimization **13**

7 Intrinsics **16**
7.1 Example: Simple matrix multiplication with intrinsics 16
7.2 Example: Large matrix multiplication with intrinsics 21

8 Program conventions: macros, types, and functions **25**
8.1 Macros 25
8.1.1 Neon macros 25
8.1.2 SVE macros 26
8.2 Data types 26
8.2.1 Neon data types 26
8.2.2 SVE data types 26
8.3 Functions 27
8.3.1 Neon functions 27
8.3.2 SVE functions 28

9 Check your knowledge **29**

10 Related information **30**

11 Next steps **31**

Copyright © 2020 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Copyright © 2020 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

1 Overview

This guide summarizes the important differences between coding for the Scalable Vector Extension (SVE) and coding for Neon. For users who have already ported their applications to Armv8-A Neon hardware, the guide also highlights the key differences to consider when porting an application to SVE.

Arm Neon technology is the Advanced Single Instruction Multiple Data (SIMD) feature for the Armv8-A architecture profile. Neon is a feature of the Instruction Set Architecture (ISA), providing instructions that can perform mathematical operations in parallel on multiple data streams.

SVE is the next-generation SIMD extension of the Armv8-A instruction set. It is not an extension of Neon, but is a new set of vector instructions that were developed to target HPC workloads. In short, SVE enables vectorization of loops which would be impossible, or not beneficial, to vectorize with Neon. Importantly, and unlike other SIMD architectures, SVE can be Vector Length Agnostic (VLA). VLA means that the size of the vector registers is not fixed. Instead, hardware implementors are free to choose the size that works best for the intended workloads.

At the end of this guide, you can Check your knowledge. You will have learned the fundamental differences between SVE and Neon, including register types, predicating instructions, and Vector Length Agnostic programming.

Copyright © 2020 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

2 Before you begin

If you are new to Arm Neon technology, read the Neon Programmer's Guide for Armv8-A for a general introduction to the subject.

If you are new to the Scalable Vector Extension (SVE), read our Arm HPC tools for SVE tutorial. This tutorial provides background information about SVE and SVE2.

Copyright © 2020 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

3 Data processing methodologies

When processing large sets of data, a major factor that limits performance is the amount of CPU time that is taken to perform data processing instructions. This CPU time depends on the number of instructions it takes to deal with the entire data set. The number of instructions depends on how many items of data each instruction can process.

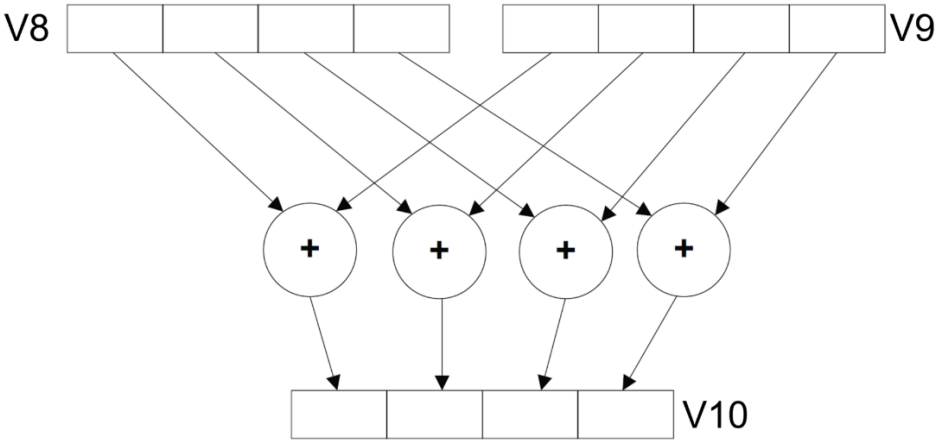
Most Arm instructions are Single Instruction Single Data (SISD). Each instruction performs one operation and writes to one output data stream. Processing multiple items requires multiple instructions.

For example, to perform four separate addition operations using traditional SISD instructions would require four instructions to add values from four pairs of registers:

```
ADD x0, x0, x5
ADD x1, x1, x6
ADD x2, x2, x7
ADD x3, x3, x8
```

Single Instruction Multiple Data (SIMD) instructions perform the same operation simultaneously for multiple items. These items are packed as separate elements in a larger register.

The following diagram shows how vector registers V8 and V9 each contain four data elements. The addition operation performs the calculation on all four lanes simultaneously, then places the results in register V10:



The following example instruction adds four pairs of single-precision (32-bit) values together. However, in this case, the values are packed as separate lanes in one pair of 128-bit registers. Each lane in the first source register is then added to the corresponding lane in the second source register, before being stored in the destination register:

```
ADD Q8.4S, Q8.4S, Q9.4S
```

Performing the four operations with a single SIMD instruction is more efficient than with four separate SISD instructions.

Copyright © 2020 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

4 Fundamentals

This section describes some of the fundamental concepts of both Neon and SVE technology.

4.1 Instruction sets

AArch64 is the name that is used to describe the 64-bit Execution state of the Armv8 architecture. In AArch64 state, the processor executes the A64 instruction set, which contains Neon instructions. These instructions are also referred to as Advanced SIMD instructions. The SVE extension is introduced in version Armv8.2-A of the architecture, and adds a new subset of instructions to the existing Armv8-A A64 instruction set.

The following table highlights the key features and instruction categories that are provided by each extension:

Extension	Key features	Categorization of new instructions
Neon	• Provides instructions that can perform mathematical operations in parallel on multiple data streams. • Supports double-precision floating-point arithmetic, enabling C code using double-precision.	• Promotion and demotion • Pair-wise operations • Load and store operations • Logical operators • Multiplication operation
SVE	• Supports wide vector and predicate registers. The introduction of predication means that instructions can be divided into two main classes: predicated and unpredicated. • Provides a set of instructions that operate on wide vectors. • Introduces minor additions to the configuration and identification registers.	• Load, store, and prefetch instructions • Integer operations • Vector address calculation • Bitwise operations • Floating-point operations • Predicate operations • Move operations • Reduction operations For descriptions of each instruction, see What is the Scalable Vector Extension?

For more information about the Neon instruction set, see the Arm A64 Instruction Set Architecture for Armv8-A.

For more information about the SVE instruction set extension, see Arm Architecture Reference Manual Supplement - The Scalable Vector Extension (SVE), for Armv8-A.

Copyright © 2020 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

4.2 Registers, vectors, lanes, and elements

Neon units operate on a separate register file of 128-bit registers and are fully integrated into Armv8-A processors. Neon units use a simple programming model. This is because they use the same address space as applications.

The Neon register file is a collection of registers. These registers can be accessed as 8-bit, 16-bit, 32-bit, 64-bit, or 128-bit registers.

The Neon registers contain vectors. A vector is divided into lanes, and each lane contains a data value called an element. All elements in a vector have the same data type.

The number of lanes in a Neon vector depends on the size of the vector and the data elements in the vector. For example, a 128-bit Neon vector can contain the following element sizes:

- Sixteen 8-bit elements
- Eight 16-bit elements
- Four 32-bit elements
- Two 64-bit elements

However, Neon instructions always operate on 64-bit or 128-bit vectors.

In SVE, the instruction set operates on a new set of vector and predicate registers: 32 Z registers, 16 P registers, and one First Faulting Register (FFR):

- The Z registers are data registers. Z register bits are an IMPLEMENTATION DEFINED multiple of 128, up to an architectural maximum of up to 2048 bits. Data in these registers can be interpreted as 8-bit, 16-bit, 32-bit, 64-bit, or 128-bit. The low 128 bits of each Z register overlap with the corresponding Neon registers, and therefore also overlap with the scalar floating-point registers.
- The P registers hold one bit for each byte that is available in a Z register. In other words, a P register is always 1/8th the size of the Z register width. Predicated instructions use a P register to determine which vector elements to process. Each individual bit in the P register specifies whether the corresponding byte in the Z register is active or inactive.
- The FFR register is a dedicated predicate register that captures the cumulative fault status of a sequence of SVE vector load instructions. SVE provides a first-fault option for some SVE vector load instructions. This option suppresses memory access faults if they do not occur as a result of the first active element of the vector. Instead, the FFR is updated to indicate which of the active vector elements were not successfully loaded.

Both the P registers and the FFR register are unique to SVE.

4.3 Vector Length Agnostic programming

SVE introduces the concept of Vector Length Agnostic (VLA) programming.

Unlike traditional SIMD architectures, which define a fixed size for their vector registers, SVE only specifies a maximum size. This freedom of choice enables different Arm architectural licensees to

Copyright © 2020 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

develop their own implementation, targeting specific workloads and technologies which could benefit from a particular vector length.

A goal of SVE is to allow the same program image to be run on any implementation of the architecture. To allow this, SVE includes instructions that permit vector code to adapt automatically to the current vector length at runtime.

For more information about VLA programming, see SVE Vector Length Agnostic programming.

Copyright © 2020 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

5 Coding best practices

As a programmer, there are various ways you can use Neon and SVE technology.

Programming in any high-level language is a tradeoff between the ease of writing code, and the amount of control that you have over the low-level instructions that the compiler outputs.

The different options available for writing code include the following:

- Neon and SVE-enabled math libraries, such as Arm Performance Libraries.


Note

An SVE-enabled library is introduced in Arm Compiler for Linux version 19.3+.

- Auto-vectorization features in your compiler can automatically optimize your code to take advantage of Neon and SVE.
- Intrinsics are function calls that the compiler replaces with appropriate Neon or SVE instructions. These intrinsics gives you direct access to the exact Neon or SVE instructions you want.

  For a searchable index for Neon intrinsics, see the Neon Intrinsics Reference.

  The SVE intrinsics are defined in the Arm C Language Extensions for SVE specification.
- For very high performance, hand-coded Neon or SVE assembly code can be an alternative approach for experienced programmers.

Copyright © 2020 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

6 Compiler optimization

Arm Compiler for Linux can automatically generate code that contains Neon and SVE instructions. Allowing the compiler to automatically identify opportunities in your code to use Neon or SVE instructions is called auto-vectorization.

Auto-vectorization includes the following specific compilation techniques:

- Loop vectorization: Unrolling loops to reduce the number of iterations, while performing more operations in each iteration
- Superword-Level Parallelism (SLP) vectorization: Bundling scalar operations together to use full width Advanced SIMD instructions

The benefits of relying on compiler auto-vectorization include the following:

- Programs implemented in high-level languages are portable, if there are no architecture-specific code elements like inline assembly or intrinsics.
- Modern compilers can perform advanced optimizations automatically.
- Targeting a given micro-architecture can be as easy as setting a single compiler option. However, hand-optimizing a program in assembly requires deep knowledge of the target hardware.

The following table shows how to compile for AArch64 with Arm Compiler for Linux:

Extension	Header file form	Recommended Arm Compiler for Linux command line	Notes
Neon	`#include <arm_neon.h>`	`armclang -O<level> -mcpu={native|<target>} -o <binary_name> <filename>.c`	To take advantage of micro-architectural optimizations, set `-mcpu` to the target processor that your application runs on. If the target processor is the same processor that you are compiling your code on, set `-mcpu=native`. This setting allows the compiler to automatically detect your processor. `-march=armv8-a` is also supported, but would not include the micro-architectural optimizations. For more information about architectural compiler flags and data to support this recommendation, see the Compiler flags across architectures: -march, -mtune, and -mcpu blog.

Copyright © 2020 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Extension	Header file form	Recommended Arm Compiler for Linux command line	Notes

SVE	`#ifdef __ARM_FEATURE_SVE`
	`#include <arm_sve.h>`
	`#endif`
		`armclang`
		`-O<level>`
		`-march=armv8-a+sve`
		`-o <binary_name>`
		`<filename>.c`
			The `-march=armv8-a+sve` option specifies that the compiler optimizes for Armv8-A hardware. You can then use Arm Instruction Emulator to emulate the SVE instructions.
			
			When SVE-enabled hardware is available and you are compiling on that target SVE hardware, Arm recommends using `-mcpu=native` instead. Using `-mcpu=native` allows you to take advantage of micro-architectural optimizations.

The following table shows the supported optimization levels for `-O<level>` for both Neon and SVE code:

Option	Description	Auto-vectorization
`-O0`	Minimum optimization for the performance of the compiled binary. Turns off most optimizations. When debugging is enabled, this option generates code that directly corresponds to the source code. Therefore, this option might result in a significantly larger image. This is the default optimization level.	Never
`-O1`	Restricted optimization. When debugging is enabled, this option gives the best debug view for the trade-off between image size, performance, and debug.	Disabled by default.
`-O2`	High optimization. When debugging is enabled, the debug view might be less satisfactory. This is because the mapping of object code to source code is not always clear. The compiler might perform optimizations that cannot be described by debug information.	Enabled by default.
`-O3`	Very high optimization. When debugging is enabled, this option typically gives a poor debug view. Arm recommends debugging at lower optimization levels.	Enabled by default.
`-Ofast`	Enable all the optimizations from level 3, including those performed with the `-ffp-mode=fast` armclang option. This level also performs other aggressive optimizations that might violate strict compliance with language standards.	Enabled by default.

Auto-vectorization is enabled by default at optimization level `-O2` and higher. The `-fno-vectorize` option lets you disable auto-vectorization.

At optimization level `-O0`, auto-vectorization is always disabled. If you specify the `-fvectorize` option, the compiler ignores it.

At optimization level `-O1`, auto-vectorization is disabled by default. The `-fvectorize` option lets you enable auto-vectorization.

Copyright © 2020 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

As an implementation becomes more complicated, the likelihood that the compiler can auto-vectorize the code decreases. For example, loops with the following characteristics are particularly difficult, or impossible, to vectorize:

- Loops with interdependencies between different loop iterations
- Loops with break clauses
- Loops with complex conditions

Neon and SVE have different conditions for auto-vectorization. For example, a necessary condition for auto-vectorizing Neon code is that the number of iterations in the loop size must be known at the start of the loop, at execution time. However, knowing the number of iterations in the loop size is not required to auto-vectorize SVE code.



Break conditions mean the loop size might not be knowable at the start of the loop, which prevents auto-vectorization for Neon code. If it is not possible to completely avoid a break condition, consider splitting the loops into multiple vectorizable and non-vectorizable parts.

You can find a full discussion of the compiler directives used to control vectorization of loops in the LLVM-Clang documentation. The two most important directives are:

- `#pragma clang loop vectorize(enable)`
- `#pragma clang loop interleave(enable)`

These pragmas are hints to the compiler to perform SLP and Loop vectorization, respectively. More detailed information about auto-vectorization is available in the Arm C/C++ Compiler and Arm Fortran Compiler Reference guides.

Copyright © 2020 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

7 Intrinsics

Intrinsics are functions whose precise implementation is known to a compiler. Intrinsics let you use Neon or SVE without having to write assembly code. This is because intrinsics themselves contain short assembly kernels, which are inlined into the calling code. Also, register allocation and pipeline optimization are handled by the compiler. This avoids many of the difficulties that often arise when developing assembly code.

Using intrinsics has several benefits:

- Powerful: With intrinsics, you have direct access to the Neon and SVE instruction sets during development. You do not need to hand-write assembly code.
- Portable: You might need to rewrite hand-written Neon or SVE assembly instructions for different target processors. You can compile C and C++ code containing Neon intrinsics for a new AArch64 target, or a new Execution state, with minimal or no code changes. However, C and C++ code containing SVE intrinsics only runs on SVE-enabled hardware.
- Flexible: You can exploit Neon or SVE only when needed. This allows you to avoid many low-level engineering concerns.

However, intrinsics might not be the right choice in all situations:

- You need more learning to use intrinsics than you need to import a library, or to rely on a compiler.
- Hand-optimized assembly code might offer the greatest scope for performance improvement, even if it is more difficult to write.

For a list of all the Neon intrinsics, see the Neon intrinsics interactive reference. The Neon intrinsics engineering specification is contained in the Arm C Language Extensions (ACLE) specification.

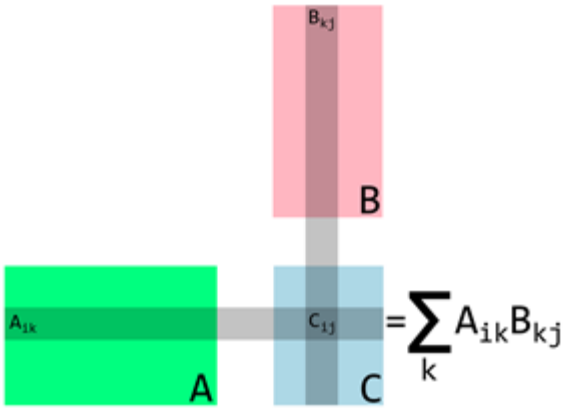
The SVE intrinsics engineering specification is contained in the Arm C Language Extensions for SVE specification.

7.1 Example: Simple matrix multiplication with intrinsics

This example implements some C functions using Neon intrinsics and using SVE intrinsics. The example does not demonstrate the full complexity of the application, but illustrates the use of intrinsics, and is a starting point for more complex code.

Matrix multiplication is an operation that is performed in many data intensive applications. Matrix multiplication consists of groups of arithmetic operations which are repeated in a simple way, as you can see in the following diagram:

Copyright © 2020 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential



Here is the matrix multiplication process:

1. Take a row in matrix A.
2. Perform a dot product of this row with a column from matrix B.
3. Store the result in the corresponding row and column of the new matrix C.

For matrices of 32-bit floats, the multiplication could be written as you can see in this code:

```
void matrix_multiply_c(float32_t *A, float32_t *B, float32_t *C, uint32_t n,
                       uint32_t m, uint32_t k) {
    for (int i_idx=0; i_idx < n; i_idx++) {
        for (int j_idx=0; j_idx < m; j_idx++) {
            C[n*j_idx + i_idx] = 0;
            for (int k_idx=0; k_idx < k; k_idx++) {
                C[n*j_idx + i_idx] += A[n*k_idx + i_idx]*B[k*j_idx + k_idx];
            }
        }
    }
}
```

Assume a column-major layout of the matrices in memory. That is, an `n` x `m` matrix `M`, is represented as an array `M_array`, where `Mij = M_array[n*j + i]`.

This code is suboptimal because it does not make full use of Neon. Intrinsics can be used to improve performance.

The following code uses intrinsics to multiply two 4x4 matrices. The loops can be completely unrolled because there is a small, fixed number of values to process. All of the values can fit into the Neon registers of the processor at the same time:

```
void matrix_multiply_4x4_neon(const float32_t *A, const float32_t *B, float32_t *C) {
    // these are the columns A
    float32x4_t A0;
    float32x4_t A1;
    float32x4_t A2;
    float32x4_t A3;

    // these are the columns B
    float32x4_t B0;
    float32x4_t B1;
    float32x4_t B2;
    float32x4_t B3;

    // these are the columns C
```

Copyright © 2020 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

```
    float32x4_t C0;
    float32x4_t C1;
    float32x4_t C2;
    float32x4_t C3;

    A0 = vld1q_f32(A);
    A1 = vld1q_f32(A+4);
    A2 = vld1q_f32(A+8);
    A3 = vld1q_f32(A+12);

    // Zero accumulators for C values
    C0 = vmovq_n_f32(0);
    C1 = vmovq_n_f32(0);
    C2 = vmovq_n_f32(0);
    C3 = vmovq_n_f32(0);

    // Multiply accumulate in 4x1 blocks, that is each column in C
    B0 = vld1q_f32(B);
    C0 = vfmaq_laneq_f32(C0, A0, B0, 0);
    C0 = vfmaq_laneq_f32(C0, A1, B0, 1);
    C0 = vfmaq_laneq_f32(C0, A2, B0, 2);
    C0 = vfmaq_laneq_f32(C0, A3, B0, 3);
    vst1q_f32(C, C0);

    B1 = vld1q_f32(B+4);
    C1 = vfmaq_laneq_f32(C1, A0, B1, 0);
    C1 = vfmaq_laneq_f32(C1, A1, B1, 1);
    C1 = vfmaq_laneq_f32(C1, A2, B1, 2);
    C1 = vfmaq_laneq_f32(C1, A3, B1, 3);
    vst1q_f32(C+4, C1);

    B2 = vld1q_f32(B+8);
    C2 = vfmaq_laneq_f32(C2, A0, B2, 0);
    C2 = vfmaq_laneq_f32(C2, A1, B2, 1);
    C2 = vfmaq_laneq_f32(C2, A2, B2, 2);
    C2 = vfmaq_laneq_f32(C2, A3, B2, 3);
    vst1q_f32(C+8, C2);

    B3 = vld1q_f32(B+12);
    C3 = vfmaq_laneq_f32(C3, A0, B3, 0);
    C3 = vfmaq_laneq_f32(C3, A1, B3, 1);
    C3 = vfmaq_laneq_f32(C3, A2, B3, 2);
    C3 = vfmaq_laneq_f32(C3, A3, B3, 3);
    vst1q_f32(C+12, C3);
}
```

Fixed size 4x4 matrices are chosen because:

- Some applications need 4x4 matrices specifically, for example graphics or relativistic physics.
- The Neon vector registers hold four 32-bit values. Matching the program to the architecture makes it easier to optimize.
- This 4x4 kernel can be used in a more general kernel.

The following table summarizes the Neon intrinsics that are used in this example:

Code element	What is it?	Why is it used?
`float32x4_t`	An array of four 32-bit floats.	One `uint32x4_t` fits into a 128-bit register and ensures that there are no wasted register bits, even in C code.

Copyright © 2020 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Code element	What is it?	Why is it used?
`vld1q_f32()`	A function which loads four 32-bit floats into `float32x4_t`.	To get the matrix values needed from A and B.
`vfmaq_lane_f32()`	A function which uses the fused multiply accumulate instruction. Multiplies a `float32x4_t` value by a single element of another `float32x4_t` then adds the result to a third `float32x4_t` before returning the result.	Since the matrix row-on-column dot products are a set of multiplications and additions, this operation fits naturally.
`vst1q_f32()`	A function which stores `float32x4_t` at a given address.	To store the results after they are calculated.

Optimizing a similar case for SVE gives the following code:

```
void matrix_multiply_nx4_neon(const float32_t *A, const float32_t *B,
                              float32_t *C, uint32_t n) {
    // these are the columns A
    svfloat32_t A0;
    svfloat32_t A1;
    svfloat32_t A2;
    svfloat32_t A3;

    // these are the columns B
    svfloat32_t B0;
    svfloat32_t B1;
    svfloat32_t B2;
    svfloat32_t B3;

    // these are the columns C
    svfloat32_t C0;
    svfloat32_t C1;
    svfloat32_t C2;
    svfloat32_t C3;

    svbool_t pred = svwhilelt_b32_u32(0, n);
    A0 = svld1_f32(pred, A);
    A1 = svld1_f32(pred, A+n);
    A2 = svld1_f32(pred, A+2*n);
    A3 = svld1_f32(pred, A+3*n);

    // Zero accumulators for C values
    C0 = svdup_n_f32(0);
    C1 = svdup_n_f32(0);
    C2 = svdup_n_f32(0);
    C3 = svdup_n_f32(0);

    // Multiply accumulate in 4x1 blocks, that is each column in C
    B0 = svld1rq_f32(svptrue_b32(), B);
    C0 = svmla_lane_f32(C0, A0, B0, 0);
    C0 = svmla_lane_f32(C0, A1, B0, 1);
    C0 = svmla_lane_f32(C0, A2, B0, 2);
    C0 = svmla_lane_f32(C0, A3, B0, 3);
    svst1_f32(pred, C, C0);

    B1 = svld1rq_f32(svptrue_b32(), B+4);
    C1 = svmla_lane_f32(C1, A0, B1, 0);
    C1 = svmla_lane_f32(C1, A1, B1, 1);
    C1 = svmla_lane_f32(C1, A2, B1, 2);
    C1 = svmla_lane_f32(C1, A3, B1, 3);
    svst1_f32(pred, C+4, C1);
```

Copyright © 2020 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

```
    B2 = svld1rq_f32(svptrue_b32(), B+8);
    C2 = svmla_lane_f32(C2, A0, B2, 0);
    C2 = svmla_lane_f32(C2, A1, B2, 1);
    C2 = svmla_lane_f32(C2, A2, B2, 2);
    C2 = svmla_lane_f32(C2, A3, B2, 3);
    svst1_f32(pred, C+8, C2);

    B3 = svld1rq_f32(svptrue_b32(), B+12);
    C3 = svmla_lane_f32(C3, A0, B3, 0);
    C3 = svmla_lane_f32(C3, A1, B3, 1);
    C3 = svmla_lane_f32(C3, A2, B3, 2);
    C3 = svmla_lane_f32(C3, A3, B3, 3);
    svst1_f32(pred, C+12, C3);
}
```

The following table summarizes the SVE intrinsics that are used in this example:

Code element	What is it?	Why is it used?
`svfloat32_t`	An array of 32-bit floats, where the exact number is defined at runtime based on the SVE vector length.	`svfloat32_t` enables you to use SVE vectors and predicates directly, without relying on the compiler for autovectorization.
`svwhilelt_b32_u32()`	A function which computes a predicate from two `uint32_t` integers	When loading from A and storing to C, `svwhilelt_b32_u32()` ensures that you do not read or write past the end of each column.
`svld1_f32()`	A function which loads 32-bit `svfloat32_t` floats into an SVE vector	To get the matrix values needed from A. This also takes a predicate to make sure that we do not load off the end of the matrix. Unpredicated elements are set to zero.
`svptrue_b32()`	A function which sets a predicate for 32-bit values to all-true	When loading from B, `svptrue_b32()` ensures that the vector fills completely. This is because the precondition of calling this function is that the matrix has a dimension which is a multiple of four.
`svld1rq_f32()`	A function which loads an SVE vector with copies of the same 128-bits (four 32-bit values)	To get the matrix values needed from B. Only loads four replicated values because the `svmla_lane_f32` instruction only indexes in 128-bit segments.
`svmla_lane_f32()`	A function which uses the fused multiply accumulate instruction. The function multiplies each 128-bit segment of an `svfloat32_t` value by the corresponding single element of each 128-bit segment of another `svfloat32_t`. The `svmla_lane_f32()` function then adds the result to a third `svfloat32_t` before returning the result.	This operation naturally fits the row-on-column dot products because they are a set of multiplications and additions.
`svst1_f32()`	A function which stores `svfloat32_t` at a given address	To store the results after they are calculated. The predicate ensures we do not store results past the end of each column.

The important difference with the SVE code is the ability to ignore one of the dimensions of the matrix because of the variable-length vectors. Instead, you can explicitly pass the length of the `n` dimension and use predication to ensure that the maximum length is not exceeded.

Copyright © 2020 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

7.2 Example: Large matrix multiplication with intrinsics

To multiply larger matrices, you can treat them as blocks of 4x4 matrices. However, this approach only works with matrix sizes which are a multiple of four in both dimensions. To use this method for other matrix sizes, you can pad the matrix with zeroes.

The following code block shows the Neon code for a more general matrix multiplication. The structure of the kernel has changed. The most important change is the addition of loops and address calculations. Like in the 4x4 kernel, unique variable names are used for the B columns. The alternative is to use one variable and reload it. This acts as a hint to the compiler to assign different registers to these variables. Assigning different registers enables the processor to complete the arithmetic instructions for one column, while waiting on the loads for another.

Here is the code:

```
void matrix_multiply_neon(const float32_t *A, const float32_t *B, float32_t *C,
uint32_t n, uint32_t m, uint32_t k) {
    /*
     * Multiply matrices A and B, store the result in C.
     * It is the user's responsibility to make sure the matrices are compatible.
     */

    int a_idx;
    int b_idx;
    int c_idx;

    // these are the columns of a 4x4 sub matrix of A
    float32x4_t A0;
    float32x4_t A1;
    float32x4_t A2;
    float32x4_t A3;

    // these are the columns of a 4x4 sub matrix of B
    float32x4_t B0;
    float32x4_t B1;
    float32x4_t B2;
    float32x4_t B3;

    // these are the columns of a 4x4 sub matrix of C
    float32x4_t C0;
    float32x4_t C1;
    float32x4_t C2;
    float32x4_t C3;

    for (int i_idx=0; i_idx<n; i_idx+=4) {
        for (int j_idx=0; j_idx<m; j_idx+=4) {
            // zero accumulators before matrix op
            C0 = vmovq_n_f32(0);
            C1 = vmovq_n_f32(0);
            C2 = vmovq_n_f32(0);
            C3 = vmovq_n_f32(0);
            for (int k_idx=0; k_idx<k; k_idx+=4){
                // compute base index to 4x4 block
                a_idx = i_idx + n*k_idx;
                b_idx = k*j_idx + k_idx;

                // load most current a values in row
                A0 = vld1q_f32(A+a_idx);
                A1 = vld1q_f32(A+a_idx+n);
                A2 = vld1q_f32(A+a_idx+2*n);
                A3 = vld1q_f32(A+a_idx+3*n);
```

Copyright © 2020 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

```
                // multiply accumulate 4x1 blocks, that is each column C
                B0 = vld1q_f32(B+b_idx);
                C0 = vfmaq_laneq_f32(C0,A0,B0,0);
                C0 = vfmaq_laneq_f32(C0,A1,B0,1);
                C0 = vfmaq_laneq_f32(C0,A2,B0,2);
                C0 = vfmaq_laneq_f32(C0,A3,B0,3);

                B1 = v1d1q_f32(B+b_idx+k);
                C1 = vfmaq_laneq_f32(C1,A0,B1,0);
                C1 = vfmaq_laneq_f32(C1,A1,B1,1);
                C1 = vfmaq_laneq_f32(C1,A2,B1,2);
                C1 = vfmaq_laneq_f32(C1,A3,B1,3);

                B2 = vld1q_f32(B+b_idx+2*k);
                C2 = vfmaq_laneq_f32(C2,A0,B2,0);
                C2 = vfmaq_laneq_f32(C2,A1,B2,1);
                C2 = vfmaq_laneq_f32(C2,A2,B2,2);
                C2 = vfmaq_laneq_f32(C2,A3,B3,3);

                B3 = vld1q_f32(B+b_idx+3*k);
                C3 = vfmaq_laneq_f32(C3,A0,B3,0);
                C3 = vfmaq_laneq_f32(C3,A1,B3,1);
                C3 = vfmaq_laneq_f32(C3,A2,B3,2);
                C3 = vfmaq_laneq_f32(C3,A3,B3,3);
            }
            // compute base index for stores
            c_idx = n*j_idx + i_idx;
            vstlq_f32(C+c_idx, C0);
            vstlq_f32(C+c_idx+n,C1);
            vstlq_f32(C+c_idx+2*n,C2);
            vstlq_f32(C+c_idx+3*n,C3);
        }
    }
}
```

Compiling and disassembling this function, and comparing it with the C function, shows the following differences:

- Fewer arithmetic instructions for a given matrix multiplication. This is because the function utilizes the Advanced SIMD technology with full register packing. Typical C code, generally, does not do this.
- `FMLA` instructions instead of `FMUL` instructions, as specified by the intrinsics.
- Fewer loop iterations. When used properly, intrinsics allow loops to be unrolled easily.

However, the disassembled code also reveals unnecessary loads and stores because of memory allocation and initialization of data types, for example, `float32x4_t`. These data types are not used in the non-intrinsics C code.

Optimizing this code for SVE produces the following code:

```
void matrix_multiply_sve(const float32_t *A, const float32_t *B, float32_t *C, uint32_t
n, uint32_t m, uint32_t k) {
    /*
     * Multiply matrices A and B, store the result in C.
     * It is the user's responsibility to make sure the matrices are compatible.
     */

    int a_idx;
```

Copyright © 2020 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

```
    int b_idx;
    int c_idx;

    // these are the columns of a nx4 sub matrix of A
    svfloat32_t A0;
    svfloat32_t A1;
    svfloat32_t A2;
    svfloat32_t A3;

    // these are the columns of a 4x4 sub matrix of B
    svfloat32_t B0;
    svfloat32_t B1;
    svfloat32_t B2;
    svfloat32_t B3;

    // these are the columns of a nx4 sub matrix of C
    svfloat32_t C0;
    svfloat32_t C1;
    svfloat32_t C2;
    svfloat32_t C3;

    for (int i_idx=0; i_idx<n; i_idx+=svcntw()) {
        // calculate predicate for this i_idx
        svbool_t pred = svwhilelt_b32_u32(i_idx, n);

        for (int j_idx=0; j_idx<m; j_idx+=4) {
            // zero accumulators before matrix op
            C0 = svdup_n_f32(0);
            C1 = svdup_n_f32(0);
            C2 = svdup_n_f32(0);
            C3 = svdup_n_f32(0);
            for (int k_idx=0; k_idx<k; k_idx+=4){
                // compute base index to 4x4 block
                a_idx = i_idx + n*k_idx;
                b_idx = k*j_idx + k_idx;

                // load most current a values in row
                A0 = svld1_f32(pred, A+a_idx);
                A1 = svld1_f32(pred, A+a_idx+n);
                A2 = svld1_f32(pred, A+a_idx+2*n);
                A3 = svld1_f32(pred, A+a_idx+3*n);

                // multiply accumulate 4x1 blocks, that is each column C
                B0 = svld1rq_f32(svptrue_b32(), B+b_idx);
                C0 = svmla_lane_f32(C0,A0,B0,0);
                C0 = svmla_lane_f32(C0,A1,B0,1);
                C0 = svmla_lane_f32(C0,A2,B0,2);
                C0 = svmla_lane_f32(C0,A3,B0,3);

                B1 = svld1rq_f32(svptrue_b32(), B+b_idx+k);
                C1 = svmla_lane_f32(C1,A0,B1,0);
                C1 = svmla_lane_f32(C1,A1,B1,1);
                C1 = svmla_lane_f32(C1,A2,B1,2);
                C1 = svmla_lane_f32(C1,A3,B1,3);

                B2 = svld1rq_f32(svptrue_b32(), B+b_idx+2*k);
                C2 = svmla_lane_f32(C2,A0,B2,0);
                C2 = svmla_lane_f32(C2,A1,B2,1);
                C2 = svmla_lane_f32(C2,A2,B2,2);
                C2 = svmla_lane_f32(C2,A3,B3,3);

                B3 = svld1rq_f32(svptrue_b32(), B+b_idx+3*k);
                C3 = svmla_lane_f32(C3,A0,B3,0);
                C3 = svmla_lane_f32(C3,A1,B3,1);
```

Copyright © 2020 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

```
                C3 = svmla_lane_f32(C3,A2,B3,2);
                C3 = svmla_lane_f32(C3,A3,B3,3);
            }
            // compute base index for stores
            c_idx = n*j_idx + i_idx;
            svst1_f32(pred, C+c_idx, C0);
            svst1_f32(pred, C+c_idx+n,C1);
            svst1_f32(pred, C+c_idx+2*n,C2);
            svst1_f32(pred, C+c_idx+3*n,C3);
        }
    }
}
```

This code is almost identical to the earlier Neon code, except for the differing intrinsics. In addition, thanks to predication, there is no longer a constraint on the number of rows of A. However, you must ensure that the number of columns of A and C, and both dimensions of B, are multiples of four. This is because the predication used in this code does not account for this. Adding further predication is possible, but would reduce the clarity of this example.

Comparing this example with the C function and Neon functions reveals that the SVE example:

- Uses `WHILELT` to determine the predicate for doing each iteration of the outer loop. This guarantees that there is always at least one element for the loop condition to process.
- Increments `i_idx` by `CNTW` (the number of 32-bit elements in a vector) to avoid hard-coding the number of elements that are processed in one iteration of the outer loop.

Copyright © 2020 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

8 Program conventions: macros, types, and functions

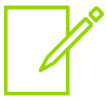
The Arm C Language Extensions (ACLE) enable C/C++ programmers to exploit the Arm architecture with minimal restrictions on source code portability. The ACLE includes a set of macros, types, and functions to make features of the Arm architecture directly available in C and C++ programs.

This section of the guides provides an overview of these features.

For more detailed information, the Neon macros, types, and functions are described in the Arm C Language Extensions (ACLE). The SVE macros, types, and functions are described in the Arm C Language Extensions for SVE specification.

8.1 Macros

The feature test macros allow programmers to determine the availability of target architectural features. For example, to use the Neon or SVE intrinsics, the target platform must support the Advanced SIMD or SVE architecture. When a macro is defined and equal to 1, the corresponding feature is available.


Note

The lists in this section are not exhaustive. Other macros are described on the Arm C Language Extensions web page.

8.1.1 Neon macros

The following table lists the macros that indicate whether particular Neon features are available or not:

Macro	Feature
`__aarch64__`	Selection of architecture-dependent source at compile time Always 1 for AArch64.
`_ARM_NEON`	Advanced SIMD is supported by the compiler. Always 1 for AArch64.
`_ARM_NEON_FP`	Neon floating-point operations are supported. Always 1 for AArch64.
`_ARM_FEATURE_CRYPTO`	Crypto instructions are available. Cryptographic Neon intrinsics are therefore available.
`_ARM_FEATURE_FMA`	The fused multiply accumulate instructions are available. Neon intrinsics which use these instructions are therefore available.

Copyright © 2020 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

8.1.2 SVE macros

The following table lists the macro that indicates whether SVE is available or not:

Macro	Feature
`_ARM_FEATURE_SVE`	Always 1 if SVE is supported. The SVE instructions are available. SVE intrinsics which use these are therefore available.

8.2 Data types

The ACLE defines several data types that support SIMD processing. These data types are different for Neon and for SVE.

8.2.1 Neon data types

For Neon, there are three main categories of data type available in `arm_neon.h`. These data types are named according to the following patterns:

Data type	Description
`<base><W>_t`	Scalar data types. For example, `int64_t`.
`<base><W>x<L>_t`	Vector data types. For example, `int32x2_t`.
`<base><W>x<L>x<N>_t`	Vector array data types. For example, `int16x4x2_t`

Where:

- `<base>` refers to the fundamental data type.
- `<W>` is the width of the fundamental type.
- `<L>` is the number of scalar data type instances in a vector data type, for example an array of scalars.
- `<N>` is the number of vector data type instances in a vector array type, for example a struct of arrays of scalars.

Generally, `<W>` and `<L>` are values where the vector data types are 64 bits or 128 bits long, and so fit completely into a Neon register. `<N>` corresponds with those instructions which operate on multiple registers at the same time.

8.2.2 SVE data types

For SVE, there is no existing mechanism that maps directly to the concept of an SVE vector or predicate. The ACLE classifies SVE vectors and predicates as belonging to a new category of type called sizeless data types. Sizeless data types are composed of vector types and predicate types, and have the prefix `sv`, for example `svint64_t`.

The following table shows the different data types that the ACLE defines:

Copyright © 2020 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Data type	Description
<base><W>_t	Scalar data types. SVE adds support for float16_t, float32_t, and float64_t.
sv<base><W>_t	Sizeless scalar data types for single vectors. For example, svint64_t.
sv<base><W>x<N>_t	Sizeless vector data types for two, three, and four vectors. For example, svint64x2_t.
svbool_t	Sizeless single predicate data type which has enough bits to control an operation on a vector of bytes.

Where:

- <base> refers to the fundamental data type.
- bool refers to the bool type from stdbool.h.
- <W> is the width of the fundamental type.
- <N> is the number of vector data type instances in a vector array type, for example a struct of arrays of scalars.

8.3 Functions

Neon and SVE intrinsics are provided as function prototypes in the header files arm_neon.h and arm_sve.h respectively. These functions follow common naming patterns.

8.3.1 Neon functions

For Neon, the function prototypes from arm_neon.h follow a common naming pattern. This is similar to the naming pattern of the ACLE.

At the most general level, this naming pattern is:

```
ret v[p][q][r]name[u][n][q][x][_high][_lane | laneq][_n][_result]_type(args)
```

For example:

```
int8x16_t vmulq_s8 (int8x16_t a, int8x16_t b)
```

The mul in the function name is a hint that this intrinsic uses the MUL instruction. The types of the arguments and the return value (sixteen bytes of signed integers) map to the following instruction:

```
MUL Vd.16B, Vn.16B, Vm.16B
```

This function multiplies corresponding elements of a and b and returns the result.

Some of the letters and names are overloaded, but the meaning of the elements in the order they appear in the naming pattern is as follows:

Pattern element	Description
ret	The return type of the function.
v	Short for vector and is present on all the intrinsics.
p	Indicates a pairwise operation. ([value] means value might be present).

Copyright © 2020 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Pattern element	Description
q	Indicates a saturating operation (except for vqtb[l][x] in AArch64 operations, where the q indicates 128-bit index and result operands).
r	Indicates a rounding operation.
name	The descriptive name of the basic operation. Often, this is an Advanced SIMD instruction, but it does not have to be.
u	Indicates signed-to-unsigned saturation.
n	Indicates a narrowing operation.
q	Post fixing the name indicates an operation on 128-bit vectors.
x	Indicates an Advanced SIMD scalar operation in AArch64. It can be one of b, h, s, or d (that is, 8, 16, 32, or 64 bits).
_high	In AArch64, used for widening and narrowing operations involving 128-bit operands. For widening 128-bit operands, high refers to the top 64 bits of the source operand (or operands). For narrowing, it refers to the top 64 bits of the destination operand.
_n	Indicates a scalar operand that is supplied as an argument.
_lane	Indicates a scalar operand taken from the lane of a vector. _laneq indicates a scalar operand that is taken from the lane of an input vector of 128-bit width. left | right means only left or right would appear.
type	The primary operand type in short form.
args	The arguments of the function.

8.3.2 SVE functions

For SVE, the function prototypes from arm_sve.h follow a common pattern. At the most general level, this is:

```
svbase[_disambiguator][_type0][_type1]...[_predication]
```

For example, svclz[_u16]_m says that the full name is svclz_u16_m and that its overloaded alias is svclz_m.

The following table describes the different pattern elements:

Pattern element	Description
Base	The lowercase name of an SVE instruction, with some adjustments
_disambiguator	Distinguishes between different forms of a function
_type0|_type1|...	Lists the types of vectors and predicates, starting with the return type and continuing with the argument types.
_predication	A suffix which describes the inactive elements in the result of a predicated operation. It can be one of z (zero predication), m (merge predication), or x ('Do not care' predication).

For more information about the individual function parts, see Arm C Language Extensions for SVE specification.

Copyright © 2020 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

9 Check your knowledge

Q. Which header files define the Neon intrinsics and the SVE intrinsics?

A. Neon intrinsics and SVE intrinsics are provided as function prototypes in the header files `arm_neon.h` and `arm_sve.h` respectively.

Q. What size are the Neon and SVE vector data registers?

A. Neon registers are 128-bit registers. SVE does not define a fixed size for its vector registers. SVE vector registers are an IMPLEMENTATION DEFINED multiple of 128 bits, up to an architectural maximum of up to 2048 bits.

Q. What term describes allowing the compiler to automatically identify opportunities in your code to use Neon or SVE instructions?

A. Auto-vectorization

Copyright © 2020 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

10 Related information

Here are some Neon resources related to material in this guide:

- Arm Neon technology
- Engineering specifications for the Neon intrinsics can be found in the Arm C Language Extensions (ACLE).
- Neon Programmer's Guide for Armv8-A
- The Architecture Exploration Tools let you investigate the Advanced SIMD instruction set.
- The Arm Architecture Reference Manual Armv8, for Armv8-A architecture profile provides a complete specification of the Advanced SIMD instruction set.
- The Neon Intrinsics Reference provides a searchable reference of the functions specified by the ACLE.

Here are some SVE resources related to material in this guide:

- Arm Architecture Reference Manual Supplement - The Scalable Vector Extension (SVE), for Armv8-A
- Arm HPC tools for SVE
- Arm Instruction Emulator
- Engineering specifications for the SVE intrinsics can be found in the Arm C Language Extensions for SVE specification.
- SVE Vector Length Agnostic programming
- The Arm HPC tools for SVE web page describes the tools to enable you to work with SVE code on AArch64.
- What is the Scalable Vector Extension?

Here are some additional resources related to material in this guide:

- Arm C/C++ Compiler Reference
- Arm Fortran Compiler Reference
- Arm Performance Libraries.
- Compiler flags across architectures: -march, -mtune, and -mcpu blog.
- LLVM-Clang documentation

Copyright © 2020 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

11 Next steps

In this guide, we provided a comparison of the important differences between coding for the Scalable Vector Extension (SVE) and coding for Neon.

If you are familiar with Neon and want to port your applications SVE, you should now understand the issues that you need to consider. For more information about the process of porting applications to Arm and optimizing for the Arm Scalar Vector Extension (SVE), see Porting and Optimizing HPC Applications for Arm SVE.

Copyright © 2020 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential